Autonomous Car Object Detection

Graduation Project - Computer Vision and Deep Learning

MLflow Experiment Report

Project	Autonomous Vehicle Object Detection using YOLOv8
Dataset	10 vehicle classes sourced from Roboflow Universe
Framework	YOLOv8 (Ultralytics) + MLflow experiment tracking
Experiments	3 runs: baseline, larger image size, larger model
Best Result	mAP50 = 0.792 achieved by Run 3 (YOLOv8s)

This report is designed to help a beginner understand how machine learning experiments are tracked, compared, and interpreted using MLflow. Each section explains a concept in clear language supported by visual charts. The goal is to prepare you to discuss this project confidently in your final presentation.

1. Experiment Parameters

We ran three training experiments. In each run we changed one or two settings called hyperparameters while keeping everything else the same. This is called controlled experimentation and it lets us understand exactly which change caused an improvement in the results.

Parameter	Run 1 - Baseline	Run 2 - imgsz=800	Run 3 - YOLOv8s
Model	YOLOv8n	YOLOv8n	YOLOv8s
Epochs	40	50	50
Image Size	640	800	640
Batch Size	16	16	8
Learning Rate	0.01	0.01	0.01
Early Stopping	No	20 epochs	20 epochs

Yellow cells indicate the parameter changed in that run.

What Each Parameter Means

Model (YOLOv8n vs YOLOv8s)

The model is the brain of the detection system. YOLOv8 comes in different sizes. The letter at the end shows the size: n = nano (smallest and fastest), s = small (bigger and smarter). A bigger model has more internal connections called neurons and can learn more complex patterns. Think of it like comparing a small calculator to a full computer.

Epochs

One epoch means the model has looked at every training image exactly once. We increased from 40 to 50 epochs in Runs 2 and 3 to give the model more learning opportunities. However more epochs is not always better because too many epochs can cause overfitting where the model memorizes training images instead of learning general patterns.

Image Size (imgsz)

This is the resolution of images fed to the model during training. A larger image size of 800 versus 640 means the model sees more pixels and more detail. This is especially helpful for detecting small objects like pedestrians and cyclists. The tradeoff is that larger images need more memory and longer training.

Batch Size

The number of images the model processes at the same time before updating its internal parameters. A smaller batch of 8 versus 16 is used in Run 3 because the larger YOLOv8s model requires more GPU memory per image.

Learning Rate (lr0)

This controls how big of a step the model takes when correcting itself. Think of it as a student correcting mistakes. A rate of 0.01 means the model makes moderate corrections. Too high and it becomes unstable. Too low and it learns very slowly.

Early Stopping (patience)

A safety mechanism that stops training automatically if the model stops improving. A patience of 20 means: if the validation score does not improve for 20 consecutive epochs, stop training. This prevents wasted time and overfitting.

2. Explanation of Each Change

Each run was designed with a specific goal. We changed only one main thing per run so we could clearly understand the effect of that change on the results.

Run 1 - Baseline

What changed: No changes. This is our starting point.

Why we made this change:

- We needed a reference point to compare all future experiments against.
- The model used was YOLOv8n (nano) — the smallest and fastest version.
- Trained for 40 epochs with standard default settings.
- This run answers: how well does a basic model perform with no optimization?

Outcome: mAP50 = 0.468. The model learned the basics but missed many detections especially small objects like pedestrians and cyclists.

Run 2 - Larger Image Size (640 to 800)

What changed: Changed imgsz from 640 to 800. Everything else stayed the same.

Why we made this change:

- Our data analysis showed that many objects in the dataset are small: pedestrians, cyclists, motorcycles.
- At imgsz=640 small objects appear as very few pixels making them hard to detect.
- Increasing to imgsz=800 means each object covers more pixels giving the model more detail.
- We also added early stopping with patience=20 to prevent overfitting.

Outcome: mAP50 = 0.791. A massive improvement of +32.3% over Run 1. The larger image size made a huge difference for small objects.

Run 3 - Larger Model (YOLOv8n to YOLOv8s)

What changed: Changed model from YOLOv8n to YOLOv8s. Image size set back to 640.

Why we made this change:

- YOLOv8s has more layers and parameters than YOLOv8n — it is a smarter model.
- More parameters means the model can learn more complex patterns in the data.
- We set imgsz back to 640 same as Run 1 to isolate the effect of model size alone.
- Batch size was reduced to 8 because YOLOv8s needs more GPU memory per image.

Outcome: mAP50 = 0.792. Slightly better than Run 2 and significantly better recall of 0.729 versus 0.686 meaning it catches more objects overall.

3. Metrics Discussion and Charts

After each training run the model is evaluated on validation images that it has never seen during training. The evaluation produces several metrics that tell us how well the model performed. Below we show how each metric changed across the 3 runs.

Results Summary

Metric	Run 1 Baseline	Run 2 imgsz=800	Run 3 YOLOv8s	Best
mAP50	0.468	0.791	0.792	**Run 3**
mAP50-95	0.339	0.608	0.599	**Run 2**
Precision	0.661	0.838	0.820	**Run 2**
Recall	0.452	0.686	0.729	**Run 3**
Box Loss	0.041	0.028	0.026	**Run 3**
Class Loss	0.023	0.015	0.013	**Run 3**

All Metrics Comparison

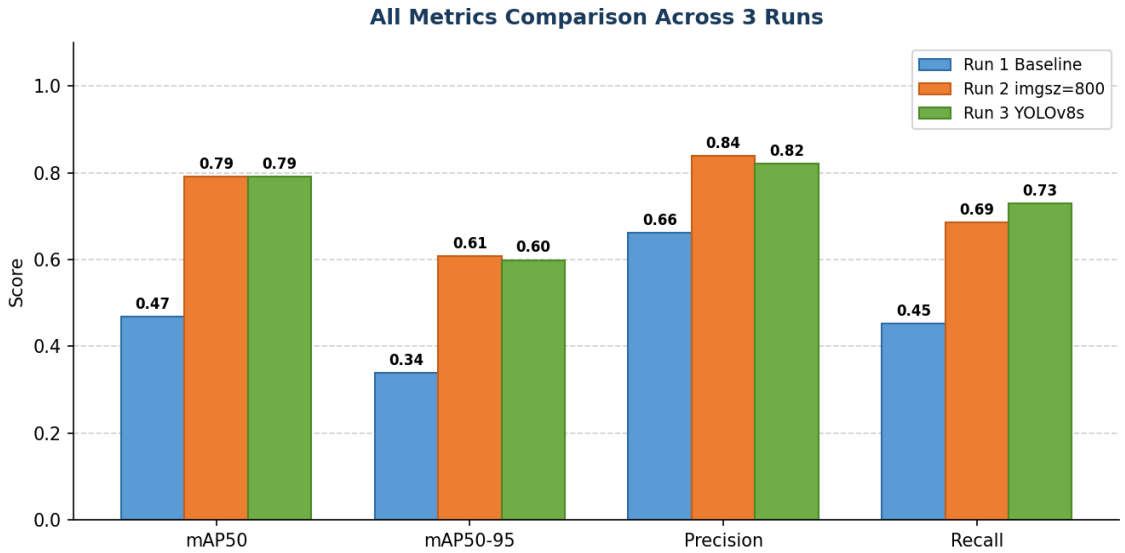

Figure 1: Grouped bar chart comparing all 4 main metrics across the 3 runs. Run 2 and Run 3 both show a dramatic improvement over Run 1.

mAP50 Comparison

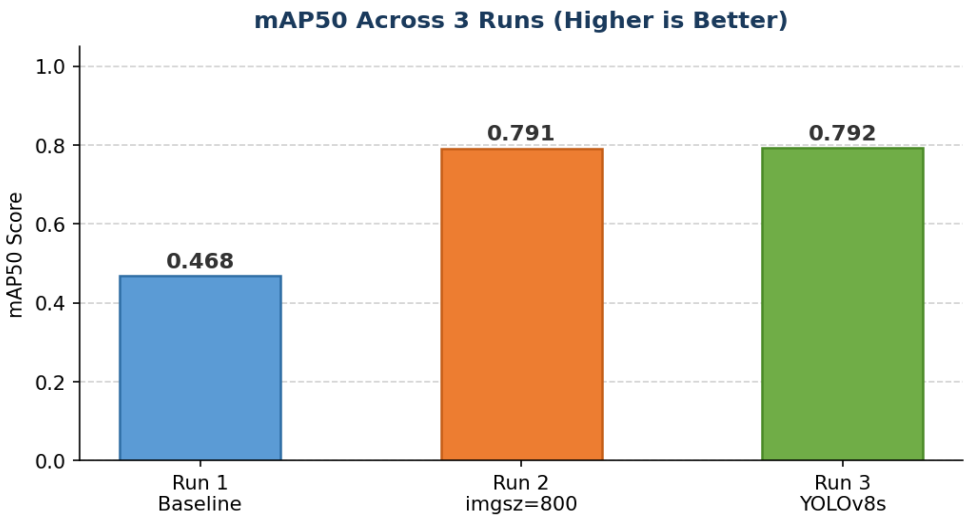

Figure 2: mAP50 scores across all 3 runs.

Recall Comparison

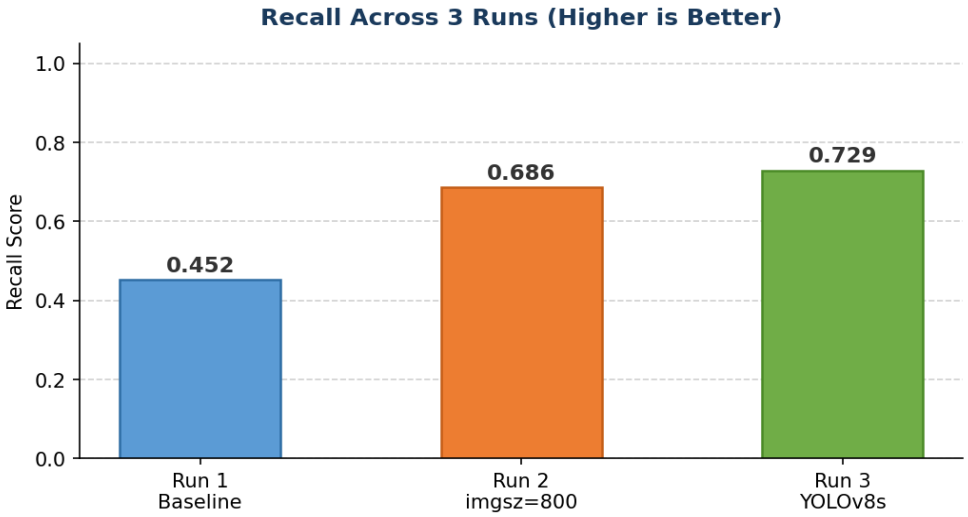

Figure 3: Recall scores showing Run 3 finds the most objects.

Training Losses Comparison

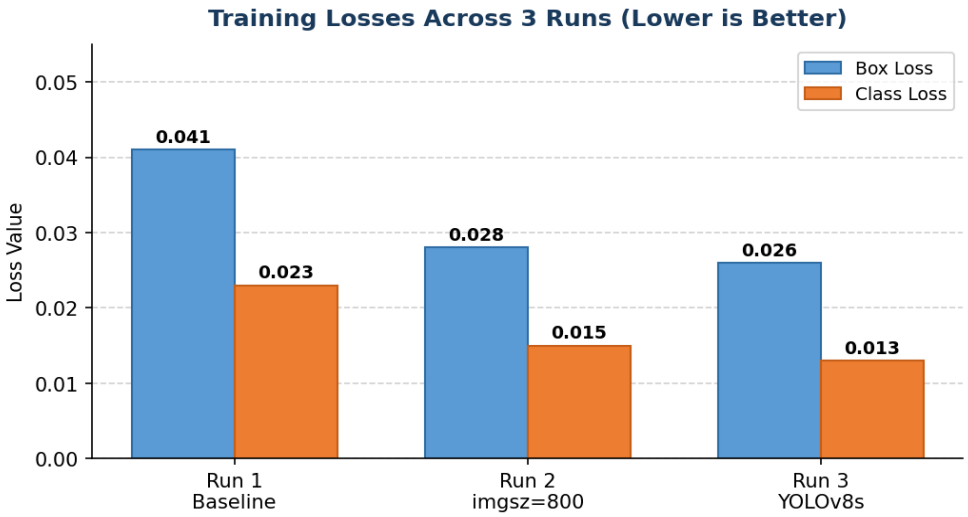

Figure 4: Box loss and class loss across 3 runs. Both losses decrease with each run showing the model is learning better.

4. Metric Terminology Explained

Below is a detailed explanation of each metric. Each one uses a simple real-world example so you can explain it confidently in your graduation discussion.

mAP50 - Mean Average Precision at IoU 0.50

In simple words: The main score for object detection. Higher is better. Maximum value is 1.0.

Technical explanation: mAP50 measures how accurately the model detects objects across all 10 classes. The number 50 means we consider a detection correct if the predicted box overlaps with the real box by at least 50 percent.

Real-world example: You draw a box around a bus on an image. The model also draws a box. If the two boxes overlap by 50 percent or more the detection is counted as correct. mAP50 averages this score across all classes and all images.

Our results: Run 1: 0.468 Run 2: 0.791 Run 3: 0.792 (best). The jump from Run 1 to Run 2 of +32.3 percent is the largest single improvement.

mAP50-95 - Mean Average Precision at Multiple Thresholds

In simple words: A stricter version of mAP50. The model must locate objects very precisely to score well.

Technical explanation: mAP50-95 calculates the average precision at 10 different overlap thresholds from 50 percent up to 95 percent. This is a much harder test because the model must place bounding boxes very accurately, not just approximately.

Real-world example: At 50 percent overlap the detection passes easily. But at 95 percent overlap the predicted box must almost perfectly match the real box. mAP50-95 averages the scores across all these thresholds.

Our results: Run 1: 0.339 Run 2: 0.608 (best) Run 3: 0.599. Run 2 slightly outperforms Run 3 meaning larger images help the model locate objects more precisely.

Precision - How Trustworthy Are the Detections?

In simple words: Of all the boxes the model drew, what percentage were correct?

Technical explanation: Precision answers: when the model says there is a pedestrian here, how often is it actually right? High precision means few false alarms.

Real-world example: The model draws 100 boxes across all test images. 83 of those boxes actually contain a real object and 17 are wrong. Precision = 83 divided by 100 = 0.83, which is Run 2's score.

Our results: Run 1: 0.661 Run 2: 0.838 (best) Run 3: 0.820. Run 2 has the fewest false detections.

Recall - How Many Objects Did the Model Find?

In simple words: Of all the real objects in the images, what percentage did the model detect?

Technical explanation: Recall answers: of all the actual pedestrians, buses, and cyclists in the test images, how many did the model find? High recall means few missed objects.

Real-world example: There are 200 real objects in test images. The model detects 146 of them and misses 54. Recall = 146 divided by 200 = 0.729, which is Run 3's score. For autonomous driving high recall is critical because missing a pedestrian is dangerous.

Our results: Run 1: 0.452 Run 2: 0.686 Run 3: 0.729 (best). The larger YOLOv8s model is best at finding hard-to-detect objects.

Box Loss - How Accurate Are the Box Locations?

In simple words: Measures how far the predicted boxes are from the real boxes. Lower is better.

Technical explanation: Box loss measures the error in the position and size of predicted bounding boxes. A lower box loss means the model is placing boxes closer to the correct location.

Real-world example: If the real box for a car starts at position 100,200 and the model predicts 110,210 there is a small positioning error. Box loss averages all these errors across all detections.

Our results: Run 1: 0.041 Run 2: 0.028 Run 3: 0.026 (best). Each run improves box accuracy.

Class Loss - How Well Does the Model Identify Object Types?

In simple words: Measures how often the model confuses one class for another. Lower is better.

Technical explanation: Class loss measures the error in classification. For example mistaking a motorcycle for a bicycle. A lower class loss means the model correctly identifies the type of object, not just its location.

Real-world example: If the model detects something correctly but calls it a Light Vehicle when it is actually a Minibus Taxi, that contributes to class loss. With 10 similar classes this is a challenging task.

Our results: Run 1: 0.023 Run 2: 0.015 Run 3: 0.013 (best). Run 3's larger model is best at distinguishing between the 10 vehicle classes.

5. Final Summary and Conclusions

After running three experiments with MLflow tracking we can draw the following conclusions about our autonomous car detection model.

Image size has the biggest impact

The jump from Run 1 to Run 2 of +32.3 percent mAP50 by simply increasing the image size from 640 to 800 was the most significant improvement. This makes sense because our dataset contains many small objects like pedestrians and cyclists that benefit greatly from higher resolution input.

Larger model improves recall

Run 3 achieved the best recall score of 0.729 by using YOLOv8s instead of YOLOv8n. The bigger model is better at finding objects that are hard to detect. This is an important quality for autonomous driving safety.

Both losses decreased consistently

Box loss and class loss decreased with each run confirming that each change helped the model learn better. Run 3 achieved the lowest losses overall showing the best learning across all runs.

Run 3 is the overall winner

Although Run 2 has slightly better precision and mAP50-95, Run 3 using YOLOv8s is chosen as the best model overall because it has the highest recall. In autonomous driving missing an object is more dangerous than a false alarm.

Motorcycles remain a challenge

During inference testing the model struggled with motorcycles. This is likely due to class imbalance in the training dataset where fewer motorcycle images exist compared to other classes. A future improvement would be to collect more motorcycle data or apply data augmentation for minority classes.

Which Run Won Each Metric?

Metric	Winner	Score	Why It Matters
mAP50	Run 3	0.792	Overall detection accuracy
mAP50-95	Run 2	0.608	Precise box localization
Precision	Run 2	0.838	Fewer false alarms
Recall	Run 3	0.729	Fewer missed objects
Box Loss	Run 3	0.026	Best box positioning
Class Loss	Run 3	0.013	Best class identification

Overall	**Run 3**	**-**	**Best overall for autonomous driving**

This report was generated as part of the autonomous car object detection graduation project. All experiments were tracked using MLflow and results are reproducible using the provided notebook: autonomous_car_detection_final_v2.ipynb